Worked for 31s

Yes — **LangGraph makes sense for ADSP**, but as an **execution/orchestration layer**, not as the methodology itself.

That distinction matters. LangGraph is explicitly aimed at long-running, stateful workflows that mix deterministic control flow with agentic behavior, with persistence and human-in-the-loop support. That maps unusually well to ADSP's staged lifecycle, stop conditions, freeze/reopen rules, and correction ledger. LangChain Ref… +1

The architecture I'd use is:

```
ADSP documents / rulebook / llms.txt
        |
        |  normative specification
        ▼
ADSP Skill
        |
        |  teaches the agent how to reason
        ▼
LangGraph workflow
        |
        |  enforces process/state transitions
        ▼
repo / experiments / tests / evidence
```

So:

> **ADSP says what good architectural reasoning is.**
> **The Skill teaches an agent the method.**
> **LangGraph makes it difficult for the agent to skip the method.**

That is stronger than making ADSP merely a Skill.

Where LangGraph helps especially well

Your ADSP already behaves like a state machine:

```
INVENTORY
   ↓
OWNERSHIP
   ↓
PREREGISTER
   ↓
HARNESS / CONTROLS
   ↓
EXPERIMENT
   ↓
INTERPRET
```

```
   ↓
PROTOTYPE
   ↓
CLOSE / SPLIT / STOP
   ↓
FREEZE
   ↓
CONSOLIDATE
   ↓
MIGRATE
   ↓
CONSUMER PROOF
   ↓
RELEASE
```

And importantly, it isn't a purely linear state machine. It has branches:

```
experiment
   |
   ├── hypothesis survives ──> next discriminator
   |
   ├── hypothesis falsified -> revise model
   |
   ├── harness invalid ──────> repair harness
   |
   ├── new question found ──> SPLIT
   |
   └── information boundary -> HARD STOP
```

That is exactly the sort of customized deterministic/agentic workflow LangGraph is designed for. LangGraph also supports checkpointed execution and human approval/interruption, so something can stop at:

```
ENTITY-DEPENDENCY-CLOSURE-0
    STATUS: HUMAN RULING REQUIRED
```

and resume later without reconstructing the project state from a giant conversation.
LangChain +1

It also answers the strongest criticism of ADSP

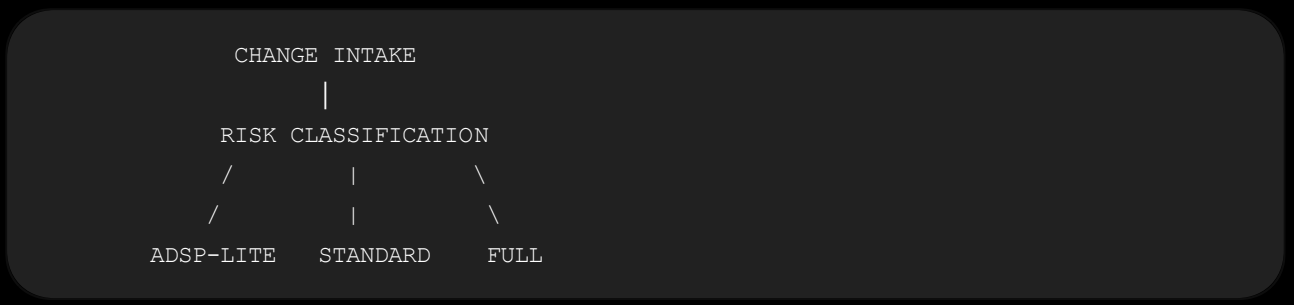
The critique's best pushback was:

> When is the full protocol overkill?

I agree with that criticism.

And I would **not** solve it with prose alone. LangGraph gives us an opportunity to encode different ADSP intensities.

Something like:



For example:

Mode	Appropriate for
Lite	internal helper, obvious implementation change, no persistent/public semantics
Standard	behavioral change with consumers, moderate ambiguity
Full	public API, persistence, transactions, identity, concurrency, restoration, migration, ownership, irreversible architecture

Then escalation rules:

```
Lite
  ↓
unexpected falsifier
  ↓
Standard

Standard
  ↓
identity / persistence / ownership ambiguity
  ↓
Full
```

That would directly address critique #1 without weakening the methodology.

And importantly:

> **Risk chooses the entry depth; evidence can always escalate the protocol.**

I think that should become an ADSP rule.

I would not let the LLM decide all gates

This is an important design constraint.

Some graph nodes can be agentic:

```
generate competing hypotheses
identify ownership candidates
propose falsifiers
interpret evidence
suggest next experiment
```

But many gates should be deterministic.

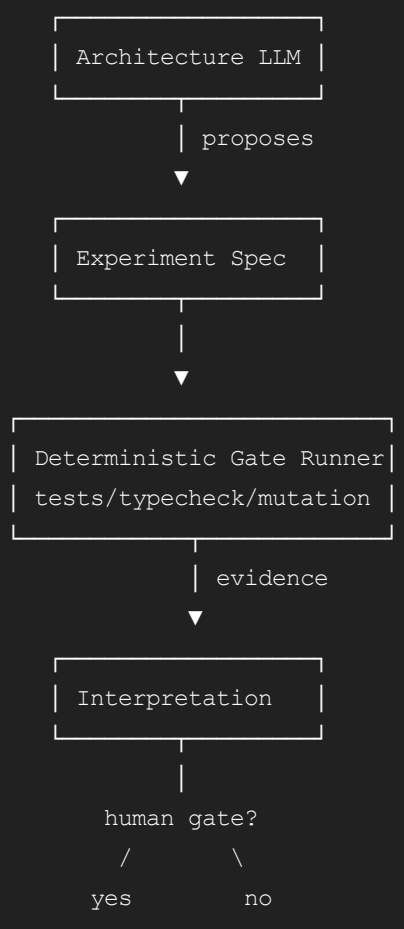
For example:

```
tests exit 0?
typecheck exit 0?
lint exit 0?
working tree clean?
mutation expected to fail actually failed?
public API diff empty?
required positive control present?
required negative control present?
```

Those should be code, not:

```
LLM: "Looks good to me."
```

The graph could therefore look like:



That fits ADSP extremely well.

The graph state should be evidence, not conversation

I wouldn't persist enormous chat histories as the core state.

I would define explicit ADSP state, perhaps conceptually:

TypeScript

```
interface AdspState {
  project: string;

  question: {
    id: string;
    statement: string;
    status:
      | 'inventory'
      | 'preregistered'
      | 'measuring'
      | 'interpreting'
      | 'stopped'
      | 'closed'
      | 'frozen';
  };

  hypotheses: Hypothesis[];

  evidence: EvidenceRef[];

  controls: {
    positive: ControlRef[];
    negative: ControlRef[];
    mechanism: ControlRef[];
    temporal: ControlRef[];
  };

  findings: Finding[];

  corrections: Correction[];

  falsifiers: Falsifier[];

  freeze?: FreezeRecord;

  openQuestions: QuestionRef[];

  commits: CommitRef[];
}
```

The actual evidence should live in the repo:

```
docs/architecture/
tests/
experiments/
commits/
ADSP ledgers
```

LangGraph state should primarily contain **references and dispositions**.

That avoids making LangGraph's persistence database the new architectural source of truth.

The correction ledger becomes especially powerful

The critique was right that the correction ledger may be the strongest artifact.

I'd make correction a first-class graph transition:

```
CLAIM
  │
  ▼
FALSIFIER
  │
  ▼
CORRECTION
  │
  ├── evidence invalid only
  │
  ├── conclusion changes
  │
  └── conclusion survives / justification replaced
```

We just had the canonical third case with rekey:

```
original:
    updateOne(id field) assumed to be rekey

falsifier:
    ids() did not move

correction:
    wrong operation

real evidence:
    changeId()

result:
    SubjectId conclusion survives
    justification replaced
```

A workflow engine could require a correction entry whenever a previously recorded claim is superseded.

That is excellent institutional memory.

It could finally demonstrate freeze reopening

The critique's point #3 is valid: we have a good rule for reopening a frozen conclusion, but not yet a clean worked example of:

```
FROZEN
  ↓
NEW FALSIFIER
  ↓
REOPENED
  ↓
REMEASURED
  ↓
REFROZEN / REPLACED
```

LangGraph could make that an explicit supported transition:

```
FROZEN ──────┐
             │ new admissible falsifier
             ▼
          REOPENED
             │
             ▼
         EXPERIMENT
```

And forbid:

```
FROZEN → casually edited because migration is inconvenient
```

That would give R-16 real enforcement rather than just prose.

It also helps with the underdeveloped performance rules

The critique about R-27/R-31 is also fair.

The graph could force a performance question into structured vectors rather than a single "faster/slower":

```
PerformanceEvidence {
    wallClock
    allocations
    retainedMemory
    asymptoticRead
```

```
    asymptoticWrite
    startup
    steadyState
    worstCase
}
```

And preregister which vectors actually matter for the experiment.

For the projection work we're doing right now:

```
entity eligible projection

retained memory:
    O(current subjects)

historical retention:
    O(0)

ordinary update:
    expected O(1)

special topology traversal:
    worst O(n)

complete Row[] materialization:
    O(n)
```

That would make R-27/R-31 much less abstract.

I would also reduce the duplication between the documents

I agree with critique #2.

I wouldn't delete the case-study narrative, but I'd make the generalized rules authoritative in ADSP and have the case study say:

```
This phase demonstrates:
    R-03
    R-05
    R-17
    R-41

Evidence:
    ...
```

rather than restating the rule.

LangGraph can even make this relationship useful: every completed experiment can record:

```
rules_exercised:
  - R-03
  - R-17
  - R-41
```

Then the case study can potentially be generated partly from the execution ledger.

That's appealing because the methodology and the receipts remain synchronized.

Skill versus LangGraph

I would actually build **both**.

A Skill alone gives you:

```
knowledge
instructions
checklists
templates
terminology
```

but the model can still skip steps.

LangGraph adds:

```
persistent workflow state
conditional branches
hard gates
human approval
resume-after-stop
explicit stage transitions
deterministic checks
```

LangGraph itself is deliberately low-level and designed for customized workflows rather than forcing a standard agent loop, which is why I think it's a better fit here than trying to shoehorn ADSP into a generic agent abstraction. LangChain +1

My recommended architecture

```
/adsp
    protocol.md            normative method
    rulebook.yaml          machine-readable laws
    templates/
    examples/

/skills/adsp/
    SKILL.md               agent operating instructions
```

```
/adsp-runtime/
    graph.ts                LangGraph orchestration
    state.ts
    gates/
        tests.ts
        mutations.ts
        git.ts
        api-diff.ts
    nodes/
        inventory.ts
        preregister.ts
        experiment.ts
        interpret.ts
        freeze.ts
        reopen.ts

/docs/adsp/
    correction-ledger.md
    freeze-ledger.md
```

I would use **LangGraph.js/TypeScript**, given SignalTree's environment, rather than introducing Python just to orchestrate the methodology.

And I would start with a tiny graph—not encode all 40+ rules on day one.

Initial proof:

```
INTAKE
  ↓
CLASSIFY RISK
  ↓
PREREGISTER
  ↓
RUN EXPERIMENT
  ↓
VALIDATE CONTROLS
  ↓
INTERPRET
  ↓
STOP / SPLIT / CLOSE
  ↓
FREEZE
```

Then see whether it actually prevents mistakes we've already encountered:

- broken positive control,
- temporal absence measured too early,
- operation didn't reach claimed mechanism,
- rendered output hid `undefined`,
- test didn't discriminate the claimed mechanism,

- frozen conclusion changed without a falsifier.

If it catches those automatically, **then** expand it.

So my recommendation is **yes: make ADSP a Skill + LangGraph workflow**, with ADSP remaining the normative source of truth. LangGraph is especially appropriate because your method is already fundamentally a persistent graph of questions, evidence, gates, stops, corrections, freezes, and reopenings—not just a prompt checklist. LangChain +1

And I agree with the critique overall. I would incorporate all four criticisms into the next ADSP revision: add graded protocol intensity, reduce case-study restatement, explicitly mark the missing freeze-reopen example, and strengthen performance examples. The critique doesn't weaken ADSP; it points mostly at **operationalization and scope control**, which is exactly where a LangGraph implementation could add the most value.